

MSMS 408 : Practical 18

Ananda Biswas

Exam Roll No. : 24419STC053

May 21, 2026

⊕ Question

- (i) Generate a sample of size 1000, each denoting the number of heads from 10 tosses of *either* a biased coin A with probability 0.4 *or* a biased coin B with probability 0.6. $P_A(\text{Head}) = \theta_A = 0.3$, $P_B(\text{Head}) = \theta_B = 0.8$ be the respective bias of the coins A and B.
Assume that you are given only h_i 's and you don't know which of the two coins each h_i comes from.
- (ii) Use Expectation-Maximization algorithm to estimate the mixing proportion, ϕ , say and the bias θ_A, θ_B .
- (iii) Show the parameter updates and increase in log-likelihood in successive iterations.
- (iv) Using the parameter estimates, predict which coin each h_i might have been generated from.

⊕ Theory

1

Data simulation :

- Generate $u_1, \dots, u_n \sim U(0, 1)$.
- Set

$$\forall i = 1(1)n, h_i = \begin{cases} x \sim \text{Binomial}(10, 0.3) & \text{if } u_i \leq 0.4 \\ x \sim \text{Binomial}(10, 0.8) & \text{if } u_i > 0.4 \end{cases}.$$

2 EM Algorithm :

- $\theta = \{\phi, \theta_A, \theta_B\}$.
- Start with an initial value $\theta^{(0)}$.
- Repeat until convergence {

(E-Step) For each i :

$$\gamma^{(i)} := P\left(z^{(i)} = A \mid h_i, \theta^{(t)}\right)$$

$$1 - \gamma^{(i)} := P\left(z^{(i)} = B \mid h_i, \theta^{(t)}\right)$$

(M-Step) Update :

$$\phi := \frac{1}{n} \sum_{i=1}^n \gamma^{(i)},$$

$$\theta_A := \frac{\sum_{i=1}^n \gamma^{(i)} h_i}{\sum_{i=1}^n \gamma^{(i)} \cdot (h_i + t_i)},$$

$$\theta_B := \frac{\sum_{i=1}^n (1 - \gamma^{(i)}) \cdot h_i}{\sum_{i=1}^n (1 - \gamma^{(i)}) \cdot (h_i + t_i)}$$

}.

3 Predict

$$x^{(i)} \in \begin{cases} \text{Coin A if } \gamma^{(i)} > 1 - \gamma^{(i)} \\ \text{Coin B if } \gamma^{(i)} \leq 1 - \gamma^{(i)} \end{cases}$$

where $\gamma^{(i)}, 1 - \gamma^{(i)}$ are evaluated on the final estimates of θ .

➔ R Program

```
1 true.theta <- c(0.4, 0.3, 0.8)
```

```
true.phi <- true.theta[1]  
true.theta.A <- true.theta[2]; true.theta.B <- true.theta[3]
```

```
x <- c()
```

```
n <- 1000
```

```
u <- runif(n)
```

```
x <- ifelse(u <= true.phi, rbinom(n, 10, true.theta.A), rbinom(n, 10, true.theta.B))
```

```
head(x)
```

```
## [1] 3 8 10 8 8 9
```

```
2 theta <- c(0.9, 0.5, 0.6)
```

```
epsilon <- 0.00001
```

```
theta.history <- list(); iter <- 1
```

```
repeat{  
  # Current Setting  
  phi <- theta[1]  
  thetaA <- theta[2]; thetaB <- theta[3]  
  
  # E-Step  
  joint.prob <- matrix(NA, nrow = n, ncol = 2)  
  joint.prob[,1] <- dbinom(x, size = 10, prob = thetaA) * phi  
  joint.prob[,2] <- dbinom(x, size = 10, prob = thetaB) * (1 - phi)  
  
  marginal.prob <- rowSums(joint.prob)  
  
  posterior.prob <- joint.prob / marginal.prob  
  
  # M-Step  
  theta.new <- c()  
  
  # update phi  
  theta.new[c(1)] <- colMeans(posterior.prob)[1]  
  
  # update thetaA  
  theta.new[c(2, 3)] <- colSums(posterior.prob * x) / colSums(posterior.prob * 10)  
  
  theta.history[[iter]] <- theta.new
```

```

iter <- iter + 1

if(max(abs(theta - theta.new)) < epsilon){
  theta <- theta.new
  break
}else{
  theta <- theta.new
}
}

```

```

theta

## [1] 0.4262251 0.2950755 0.8017629

```

3

```

df1 <- data.frame(do.call(rbind, theta.history))
colnames(df1) <- c("phi", "theta.A", "theta.B")
rownames(df1) <- paste("Iteration ", 1:(iter-1))

```

Table 1: Estimates in Successive Iterations

	phi	theta.A	theta.B
Iteration 1	0.839103	0.546305	0.791771
Iteration 2	0.724858	0.478870	0.867506
Iteration 3	0.622958	0.417766	0.863429
Iteration 4	0.551534	0.374707	0.845408
Iteration 5	0.502562	0.343931	0.830160
Iteration 6	0.471302	0.323857	0.819307
Iteration 7	0.452258	0.311592	0.812207
Iteration 8	0.441052	0.304416	0.807833
Iteration 9	0.434603	0.300324	0.805237
Iteration 10	0.430938	0.298016	0.803732
Iteration 11	0.428869	0.296721	0.802873
Iteration 12	0.427705	0.295995	0.802386
Iteration 13	0.427051	0.295588	0.802111
Iteration 14	0.426684	0.295361	0.801957
Iteration 15	0.426479	0.295233	0.801870
Iteration 16	0.426364	0.295161	0.801821
Iteration 17	0.426299	0.295121	0.801794
Iteration 18	0.426263	0.295099	0.801779
Iteration 19	0.426243	0.295086	0.801770
Iteration 20	0.426232	0.295079	0.801766
Iteration 21	0.426225	0.295075	0.801763

```

log.likelihood.single <- function(theta, h){
  a <- theta[1] * dbinom(h, size = 10, prob = theta[2])
  b <- (1 - theta[1]) * dbinom(h, size = 10, prob = theta[3])
}

```

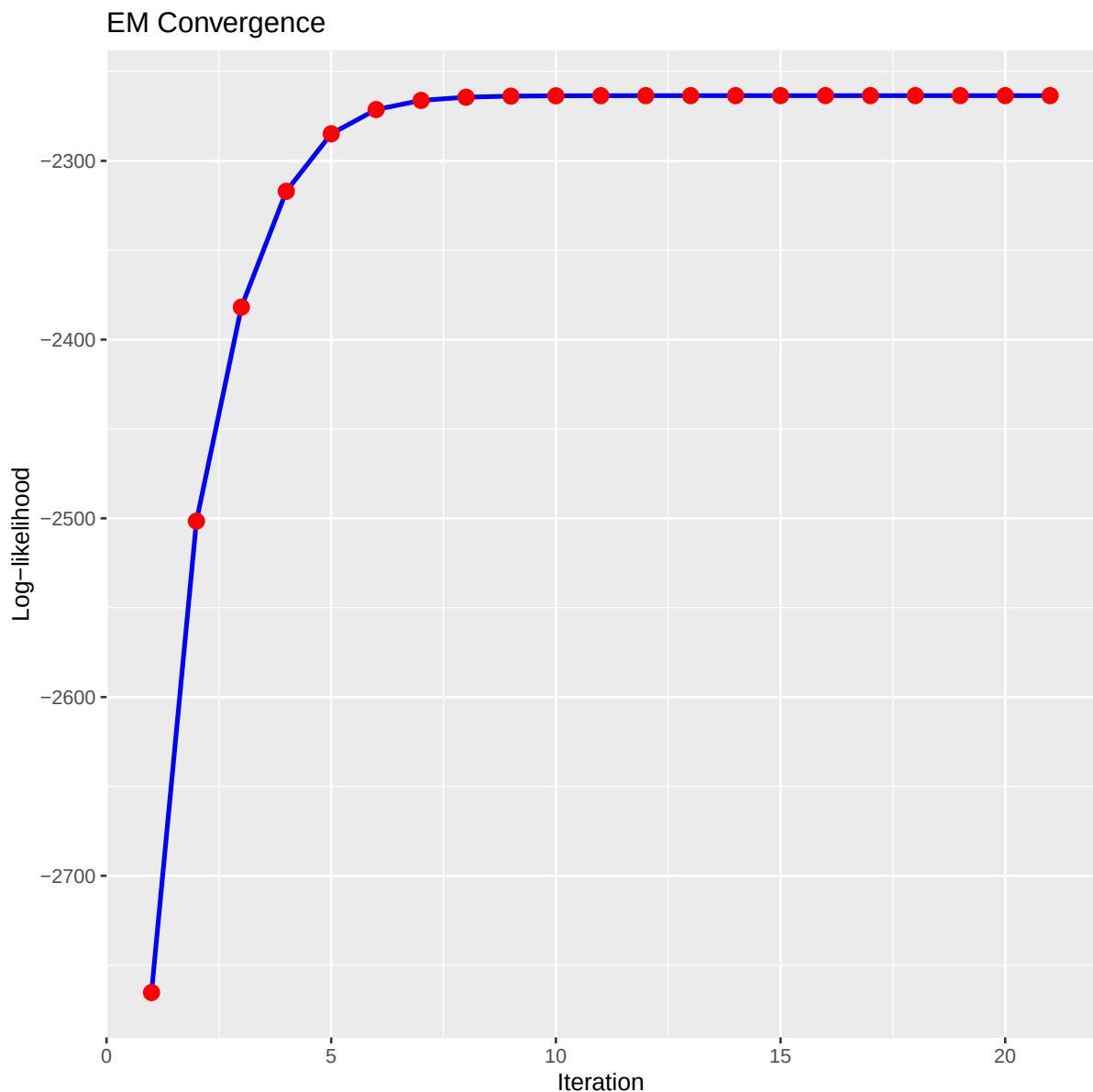
```
    return(log(a+b))  
  }
```

```
log.likelihood <- function(theta) sum(log.likelihood.single(theta, x))
```

```
log.likelihood.history <- apply(df1, 1, log.likelihood)
```

```
df2 <- data.frame(iteration = 1:(iter-1),  
                  log.likelihood = log.likelihood.history)
```

```
df2 %>%  
  ggplot(aes(x = iteration, y = log.likelihood)) +  
  geom_line(linewidth = 1, col = "blue") +  
  geom_point(size = 3, col = "red") +  
  labs(x = "Iteration", y = "Log-likelihood", title = "EM Convergence")
```



- 4 We now calculate the posterior probabilities using the latest setting of the parameters, hence provide an estimate of which Gaussian each h_i might belong to.

```
phi <- theta[1]
thetaA <- theta[2]; thetaB <- theta[3]
```

```
joint.prob <- matrix(NA, nrow = n, ncol = 2)
joint.prob[,1] <- dbinom(x, size = 10, prob = thetaA) * phi
joint.prob[,2] <- dbinom(x, size = 10, prob = thetaB) * (1 - phi)
```

```
marginal.prob <- rowSums(joint.prob)
```

```
posterior.prob <- joint.prob / marginal.prob
```

```
head(posterior.prob)
```

```
##           [,1]      [,2]
## [1,] 9.962580e-01 0.00374204
## [2,] 3.151660e-03 0.99684834
## [3,] 3.386525e-05 0.99996613
## [4,] 3.151660e-03 0.99684834
## [5,] 3.151660e-03 0.99684834
## [6,] 3.271128e-04 0.99967289
```

```
assignment <- data.frame(sample = x,
                          predicted.coin = ifelse(max.col(posterior.prob) == 1,
                                                    "A", "B"))
```

Table 2: Assignment to Coins

sample	predicted.coin
3	A
8	B
10	B
8	B
8	B
9	B
9	B
8	B
7	B
1	A

➔ Conclusion

Parameter	Estimate	True Value
$\hat{\phi}$	0.426	0.4
$\hat{\theta}_A$	0.295	0.3
$\hat{\theta}_B$	0.802	0.8

Table 3: Estimated and true parameter values

 Even though EM algorithm has a tendency to converge to the local optima, it has performed considerably well in our problem.